

Predictive Modelling Report: NBA Score Analysis

Léandre Javelot

March 6 – March 13, 2026

Contents

1 Introduction	1
2 Feature Selection with LASSO	2
2.1 Feature Pre-selection	2
2.2 Justification and Presentation of LASSO Penalization	2
2.3 Results	3
3 Random Forests	4
3.1 Justification	4
3.2 Results	4
4 Conclusion and Limitations	5

1 Introduction

This document presents two approaches to predicting team point distributions in the NBA. They rely on processing approximately 15,000 historical NBA match statistics from the year 2020.

The original dataset provides 125 meta/macro statistics for each match. To predict the distribution for a given match, both approaches process the historical statistics of the home and away teams. These past statistics are calculated by averaging data over different rolling windows.

The goal of the first approach is to select important features and provide a team point distribution. Assuming linearity between features and team points is a simple and effective starting point. A well-known model for this is LASSO regression, which serves as our first approach.

Building on the important features identified in a linear framework, the second approach aims to capture non-linearities and provide a more accurate team point distribution. Random Forests are an excellent method to achieve this.

The primary objective throughout this task is to evaluate the models, i.e., to determine whether they outperform naive predictions (which simply assume a point distribution based on basic assumptions). Thus, the goal is to measure the extent to which these models overperform naive ones. As a performance metric, we use the Ranked Probability Score (RPS), which is highly regarded in the sports industry.

To maintain flexibility, all models are built from scratch without using scikit-learn. This also reflects how I learned data science academically at Ensimag and Imperial College London.

2 Feature Selection with LASSO

2.1 Feature Pre-selection

The goal of this section is to extract important features for score distribution prediction. As mentioned, we use past statistics for each team over different rolling windows. Given the arbitrary rolling windows of 3, 5, and 10 previous matches, the total feature set to shrink would be approximately $125 \times 3 = 375$. However, one can easily and drastically reduce this amount using common sense. With a little sports intuition, one can anticipate which variables are relevant to consider when predicting team scores. For example, average team points in past matches are likely more relevant than the number of isolation rim attempts.

The feature pre-selection process can be found in the notebook linked with this document.

2.2 Justification and Presentation of LASSO Penalization

LASSO has two main advantages for predictive modelling in sports:

- It performs feature selection, providing interpretability and reducing correlations.
- Even though it adds bias to the point estimator, it lowers its variance. This means that if one uses two slightly different training sets to build two models, the resulting models will remain close to each other. In other words, the model is less sensitive to the intrinsic, highly noisy nature of sports statistics. As long as the bias is controlled, this is a desirable property.

Given the random variable Y representing team points, and the matrix $\mathbf{X}$ of past team statistics, the linear regression problem can be written as follows:

$$Y = \mathbf{X}^T \boldsymbol{\beta} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, I\sigma^2)$$

where $\mathcal{N}(0, I\sigma^2)$ is a multivariate normal distribution with mean 0 and covariance matrix $I\sigma^2$. The fact that the covariance matrix is a multiple of the identity matrix suggests no correlation between the noise of different matches, which is a reasonable assumption. Even if there were noise correlation, since we use statistics based on past matches, the model intrinsically accounts for noise correlation between matches of the same teams, minimizing the issue.

Thus, under this model, the assumed team point distribution for a match with statistics x is $\mathcal{N}(x\boldsymbol{\beta}, \sigma^2)$, and its estimate will be $\mathcal{N}(x\hat{\boldsymbol{\beta}}, \hat{\sigma}^2)$, where $\hat{\sigma}^2$ is the Mean Squared Error (MSE) of the residuals.

The fact that the distribution of team points is continuous rather than discrete is a theoretical disadvantage, but we maintain this assumption for now.

In LASSO regression, the aim is to minimize the following loss function:

$$L_{\text{LASSO}}(\boldsymbol{\beta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \lambda\|\boldsymbol{\beta}\|_1$$

where

$$\|\boldsymbol{\beta}\|_1 = \sum_{i=1}^p |\beta_i| \quad (\text{Hence the feature selection}).$$

The gradient of this loss is:

$$\nabla_{\boldsymbol{\beta}} L_{\text{LASSO}}(\boldsymbol{\beta}) = 2(\mathbf{X}^T \mathbf{X}\boldsymbol{\beta} - \mathbf{X}^T \mathbf{y}) + \lambda \text{sign}(\boldsymbol{\beta})$$

This equation has no analytical solution (it cannot simply be set to zero), so the gradient descent algorithm is used to perform the minimization:

$$\boldsymbol{\beta}_{t+1} = \boldsymbol{\beta}_t - \eta_t \nabla_{\boldsymbol{\beta}} L_{\text{LASSO}}(\boldsymbol{\beta}_t)$$

where η_t is the step size (learning rate) of the descent.

To determine the optimal λ for the model, we perform K-fold cross-validation (respecting time order to avoid data leakage) using the Ranked Probability Score (*RPS*) as the loss function.

For a single basketball match with K possible ordered outcomes, the *RPS* is defined as:

$$RPS = \sum_{k=1}^K (P_k - O_k)^2$$

This can be expanded using the individual probability mass functions:

$$RPS = \sum_{k=1}^K \left(\sum_{j=1}^k p_j - \sum_{j=1}^k o_j \right)^2$$

where K is the total number of possible outcomes (here, the possible points scored, assumed to range from 0 to 200); p_j is the predicted probability that the exact outcome is j ; o_j is the actual observed outcome (equal to 1 if the actual score is j , and 0 otherwise); and P_k and O_k are the cumulative probabilities of the forecast and the observation, respectively, at threshold k . (Thus, $\exists k^* \in \{1, \dots, K\}$ such that $\forall k, O_k = \mathbb{1}_{\{k \geq k^*\}}$, meaning O is a Heaviside step function).

When evaluating the model (first during cross-validation, then on the test set), we average the *RPS* over the samples (matches):

$$\overline{RPS} = \frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K (P_{i,k} - O_{i,k})^2$$

where N is the number of predicted team point distributions (i.e., the number of predicted matches).

For cross-validation, we use the mean of the averaged RPS across the folds.

The RPS offers two main advantages:

- It evaluates the entire probability distribution rather than a single point estimate, rewarding models that accurately quantify their uncertainty and are well-calibrated.
- It respects the ordinal nature of the outcomes by measuring the "distance" of an error. Because it uses cumulative probabilities, predicting 109 points when the true score is 110 is penalized much less than predicting 80 points, which is highly relevant for sports forecasting.

To evaluate the true contribution of the algorithms, analyzing the raw RPS score alone is insufficient, as its absolute value is difficult to interpret in isolation. The "Skill Score" quantifies the percentage improvement of a predictive model compared to a reference forecast. Here, the naive forecast predicts the point distribution as a discretized normal distribution, with a mean equal to the team's average points over the past 10 matches, and a variance equal to the mean squared error of past residuals.

$$RPSS = 1 - \frac{\overline{RPS}_{model}}{\overline{RPS}_{naive}}$$

2.3 Results

The 7-fold cross-validation identified the optimal penalty at $\lambda = 6773$.

At this λ , the selected important features are:

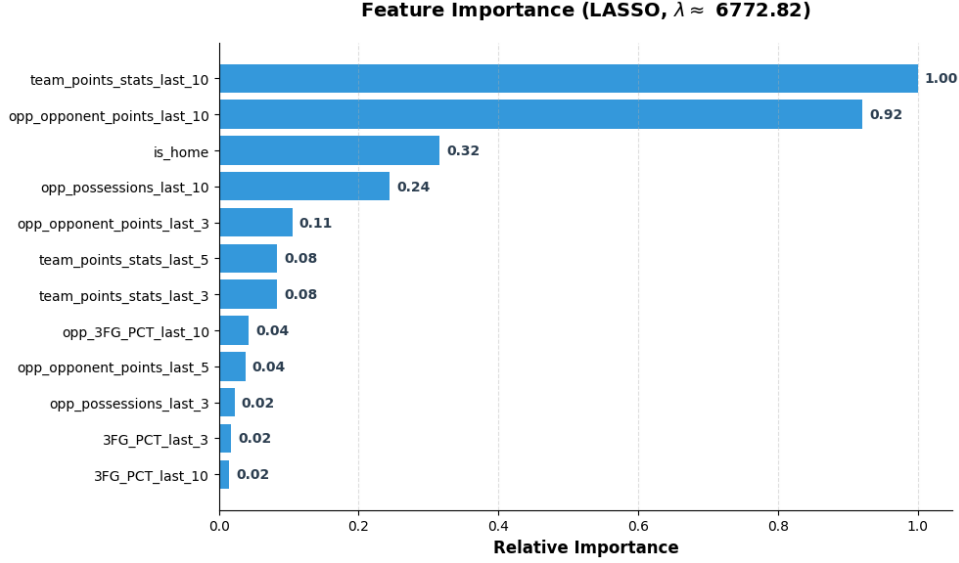


Figure 1: Feature importance plot

Unsurprisingly, the most important feature is the average team points over past matches, which reflects the team’s offensive strength. The second most important feature evaluates the weakness of the opponent’s defense. The third is whether the team in question is playing at home or not, which is a well-known critical factor.

Finally, when computing RPS_{model} and RPS_{naive} on the test set, we obtain the following $RPSS$:

$$RPSS = 6.61\%$$

Such a positive $RPSS$ means that the model provides better distributions (in terms of RPS comparison) than the naive one. A score above 5% suggests solid performance.

3 Random Forests

3.1 Justification

A limitation of LASSO is the linearity assumption, which can be restrictive. Furthermore, the primary aim of such regressions is to predict a single point estimate, not a distribution. To address this, we can implement decision trees to produce empirical distributions of team points, trained by minimizing the variance of each distribution. Because decision trees tend to overfit, we mitigate this by implementing Random Forests. This method trains multiple trees by randomly sampling both the training set (bootstrapping) and the features on which to split the data (feature bagging).

As with LASSO, we perform cross-validation using RPS to determine the optimal value for a key hyperparameter: the minimum standard deviation σ_{min} of team points within the leaves of the trees. This means we stop splitting a node when this standard deviation threshold is reached.

3.2 Results

The cross-validation provides the following plot:

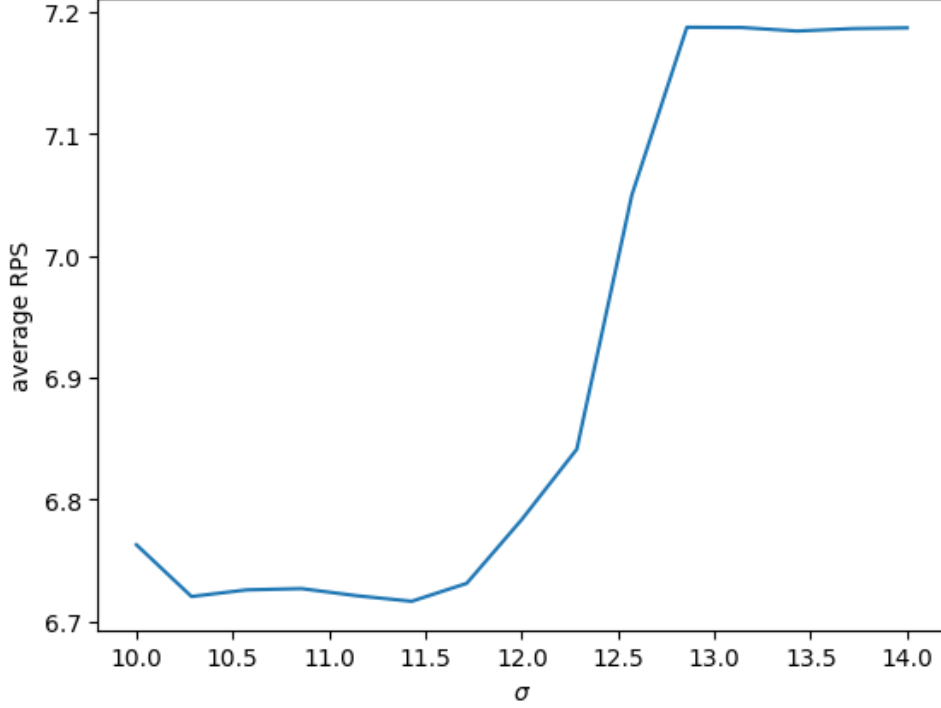


Figure 2: Feature importance plot

The cross-validation indicates an optimal σ of around 11.5.
Comparing this against the naive model on the test set, we obtain:

$$RPSS = 5.91\%$$

4 Conclusion and Limitations

The most significant limitation here was the computational power required to build the forests (even with thread parallelization implemented in the notebook, it took 1 hour to produce the plot above). With custom-built Python algorithms, achieving fast processing times is difficult. In the future, one could build models using scikit-learn or utilize Gradient Boosting Decision Trees instead of Random Forests.

Additionally, we could apply a Gaussian kernel to avoid scenarios where certain team scores close to the average are assigned a 0 probability in some of the final tree nodes (which is theoretically possible).

It is not entirely fair to criticize the lower RPSS score (which is still better than the naive approach) without considering the full potential of decision trees, which were not fully explored here.

In conclusion, the LASSO model proves to be an effective approach for both predicting distributions and identifying feature importance.

```
In [15]: import pandas as pd
import numpy as np
import random as rd
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
```

First, we prepare data

```
In [2]: df=pd.read_csv('nba_dataset.csv')
display(df.head())
print(f'Total rows : {df.shape[0]},\nTotal columns : {df.shape[1]}')
df=df.sort_values('match_date')

df['team_points_stats'] = df['team_points']
colonnes_meta = [
    'game_id',
    'match_date',
    'team_id',
    'team_name',
    'team_points',
    'is_home'
]

colonnes_stats = [
    'team_points_stats',
    'ORTG',
    'DRTG',
    'possessions',
    'FG_PCT',
    '3FG_PCT',
    'FTA',
    'turnovers',
    'OREB',
    'opponent_points',
]

df=df[colonnes_meta+colonnes_stats]

rolling_windows=[3,5,10]
df_grouped=df.groupby('team_name')
for feature in colonnes_stats:
    for rolling_window in rolling_windows:
        col_name = f"{feature}_last_{rolling_window}"
        try :
            df[col_name] = df_grouped[feature].transform(
                lambda x: x.rolling(window=rolling_window).mean().shift(1)
            )
        except :
            print(col_name)
            pass
```

	Unnamed: 0	_id	tournament_id	season_id	game_id	match_date	team_id	team_name
0	10696	2-615296	3	26	615296	2020-12-23 03:00:00	2	Gol S Warr
1	10697	3223-615296	3	26	615296	2020-12-23 03:00:00	3223	Broo ↑
2	10698	13-615303	3	26	615303	2020-12-23 06:00:00	13	Los Ang Clipi
3	10699	3171-615303	3	26	615303	2020-12-23 06:00:00	3171	Los Ang La
4	10700	1-617993	3	26	617993	2020-12-24 03:00:00	1	Clevel Cava

5 rows × 125 columns



Total rows : 14746,
Total columns : 125

Then, we have to add opponent team stat on each row.

```
In [3]: rolling_features = [col for col in df.columns if '_last_' in col] + ['is_home']
opponent_features = ['game_id', 'team_id', 'team_points'] + rolling_features

df_away = df[opponent_features].copy()
df_home = df[opponent_features].copy()

renaming_dict = {col: f"opp_{col}" for col in df_away.columns if col != 'game_id'}
df_away = df_away.rename(columns=renaming_dict)

df_merged = pd.merge(df_home, df_away, on='game_id') #first merge

df_final = df_merged[df_merged['team_id'] != df_merged['opp_team_id']].reset_index(drop=True)
df_final_without_na = df_final.dropna().reset_index(drop=True)
```

```
In [4]: statistics_names = [col for col in df_final.columns if '_last_' in col] + ['is_h

y=df_final_without_na['team_points'].values
x=np.array(df_final_without_na[statistics_names])

print(f"Shape of X : {x.shape}")
print(f"Shape of Y : {y.shape}")
```

Shape of X : (14426, 61)
Shape of Y : (14426,)

Then, we build training and test sets. It is important to keep time order between training and testing to avoid data leakage.

```
In [5]: ratio=0.8
index_max=int(ratio*x.shape[0])
```

```

x_train=x[:index_max,:]
colonne_ones_train = np.ones((x_train.shape[0], 1))

x_test=x[index_max:,:]
colonne_ones_test= np.ones((x_test.shape[0], 1))

y_train=y[:index_max,]
y_test=y[index_max:,:]

```

We will need to standardize data (with train set data only to avoid data leakage)

```

In [6]: def standardize(x_train,x):
        mean=np.mean(x_train,axis=0)
        sigma=np.std(x_train,axis=0)
        x_standardized=(x-mean)/sigma
        return x_standardized

```

```

In [7]: x_train_standardize=standardize(x_train,x_train)
        x_test_standardize=standardize(x_train,x_test)

#making sure x_train is standardized

assert np.allclose(np.mean(x_train_standardize,axis=0),np.zeros(x_train.shape[1]
assert np.allclose(np.std(x_train_standardize,axis=0),np.ones(x_train.shape[1]))

colonne_ones_test= np.ones((x_test.shape[0], 1)) #for offset
colonne_ones_train = np.ones((x_train.shape[0], 1))
x_train_augmente = np.hstack((colonne_ones_train, x_train_standardize))
x_test_augmente = np.hstack((colonne_ones_test,x_test_standardize))

```

Here we define gradient descent to minimize the loss

```

In [8]: def grad_lasso(beta):
        return np.sign(beta)

```

```

In [9]: def minimize_ls_lasso(X, y, lambd, n_iters = 15000, step_size=1e-7, c_huber=1e-5
        """
        This function estimates the regression parameters using the gradient-descent
        solution.
        Args:
        X (np.array): The augmented data matrix with shape (N, p + 1).
        y (np.array): The response column with shape (N, 1).
        lambd (float): The multiplier of the relaxed L1 term.
        n_iters (int): Number of gradient descent iterations.
        step_size (float): The step size in the updating step.
        """

        # check X and y have the same Length
        assert X.shape[0] == y.shape[0], "Input X and y have different lengths."

        N, p = X.shape
        # Precomputed products to avoid redundant computations.
        XX = X.T @ X
        Xy = X.T @ y
        # Initialize beta params with zeros

```



```

beta = np.zeros(p)

for _ in range(n_iters):

    grad_l= grad_lasso(beta)

    # Intercept term is not involved in the regularisation.
    grad_l[0]=0

    # Compute the gradient of the regularised loss.
    grad = 2*(XX @ beta - Xy) + lambd * grad_l

    # Update beta
    beta = beta - step_size * grad

return beta

```

To find the best lambda, we proceed a T-fold cross validation, but we don't sample folds at random as we need to keep time order to avoid data leakage in training. Here, each fold is included in the following, starting from the beginning of time, and the validation set is the following set in time.

As a loss function to minimize, we use the ranked probability score (RPS), which compare two discrete cumulative distribution function. It is well known in sports industry.

```

In [10]: def ranked_probability_score(predictions, y_true, support):
        """
        Compute average RPS
        """
        n_samples = predictions.shape[0]
        n_classes = len(support)

        cdf_preds = np.cumsum(predictions, axis=1)
        cdf_true = np.zeros((n_samples, n_classes))

        for i in range(n_samples):
            true_index = np.where(support == y_true[i])[0][0]
            cdf_true[i, true_index:] = 1.0

        rps_per_sample = np.sum((cdf_preds - cdf_true)**2, axis=1)

        return np.mean(rps_per_sample)

```

However, we need to discretise normal distributions as it is the underlying model in linear regressions. To do so, we use the module scipy which provide the cdf of normal.

```

In [11]: from scipy.stats import norm

def discretize_normal_cdf(mu_vector, sigma, support_max=200): #we suppose points

    mu_col = np.asarray(mu_vector).reshape(-1, 1)

    support_row = np.arange(support_max + 1).reshape(1, -1)

```

```

upper_bounds = support_row + 0.5
lower_bounds = support_row - 0.5

prob_matrix = norm.cdf(upper_bounds, loc=mu_col, scale=sigma) - norm.cdf(lower_bounds, loc=mu_col, scale=sigma)

prob_matrix = prob_matrix / np.sum(prob_matrix, axis=1, keepdims=True)

return prob_matrix

```

Then, we can write the cross validation

```

In [12]: # the variance estimator is the MSE of residuals
def mse(y_predicted, y_true):
    return np.mean((y_predicted - y_true)**2)

def T_fold_cross_validation_time(x_train, y_train, subsets_indexes, lamb) :
    errors=[]
    for k in range(1, len(subsets_indexes)-1):
        subset_index=subsets_indexes[k]
        next_subset_index=subsets_indexes[k+1]
        x_train_t=x_train[:subset_index,:]
        y_train_t=y_train[:subset_index]
        x_val=x_train[subset_index:next_subset_index,:]
        y_val=y_train[subset_index:next_subset_index]
        betas_t = minimize_ls_lasso(
            x_train_t,
            y_train_t,
            lambd=lamb,
            n_iters=15000,
            step_size=1e-7,
            c_huber=1e-5
        )
        y_predicted=x_val@betas_t
        error=mse(y_predicted, y_val)
        predicted_distribution=discretize_normal_cdf(y_predicted, np.sqrt(error))
        score=ranked_probability_score(predicted_distribution, y_val, np.arange(20))
        errors.append(score)

    return np.mean(errors)

```

We can finally find the best lambda with cross validation, and plot the loss function as a function of $\log(\lambda)$

```

In [13]: def record_loss_curve_cross_validation(x_train, y_train, hyperparameter_list, cross
losses=[]
best_loss=np.inf
iterations_number=hyperparameter_list.shape[0]
iteration=0
for hyperparameter in hyperparameter_list :
    iteration+=1
    loss=cross_validation_function(x_train, y_train, subsets_indexes, hyperparameter)
    if loss < best_loss :
        best_hyperparameter, best_loss=hyperparameter, loss

    print(f'Test for {hp_name} = {hyperparameter:.4f}, average RPS = {loss:.4f}')
    losses.append(loss)

```

```

print('\n')
print(f'Best error : {best_loss:.4f}, Best {hp_name} : {best_hyperparameter:}
return best_hyperparameter,losses

```

```

In [16]: folds_number=7
subsets_indexes=np.linspace(0,x_train_augmente.shape[0],folds_number,dtype=int)
lambdas=np.logspace(2.6,4.6,40)
best_lambda,losses = record_loss_curve_cross_validation(x_train_augmente,y_train
plt.plot(np.log10(lambdas),losses)
plt.xlabel(f'log($\lambda$)')
plt.ylabel("average RPS")
plt.show()

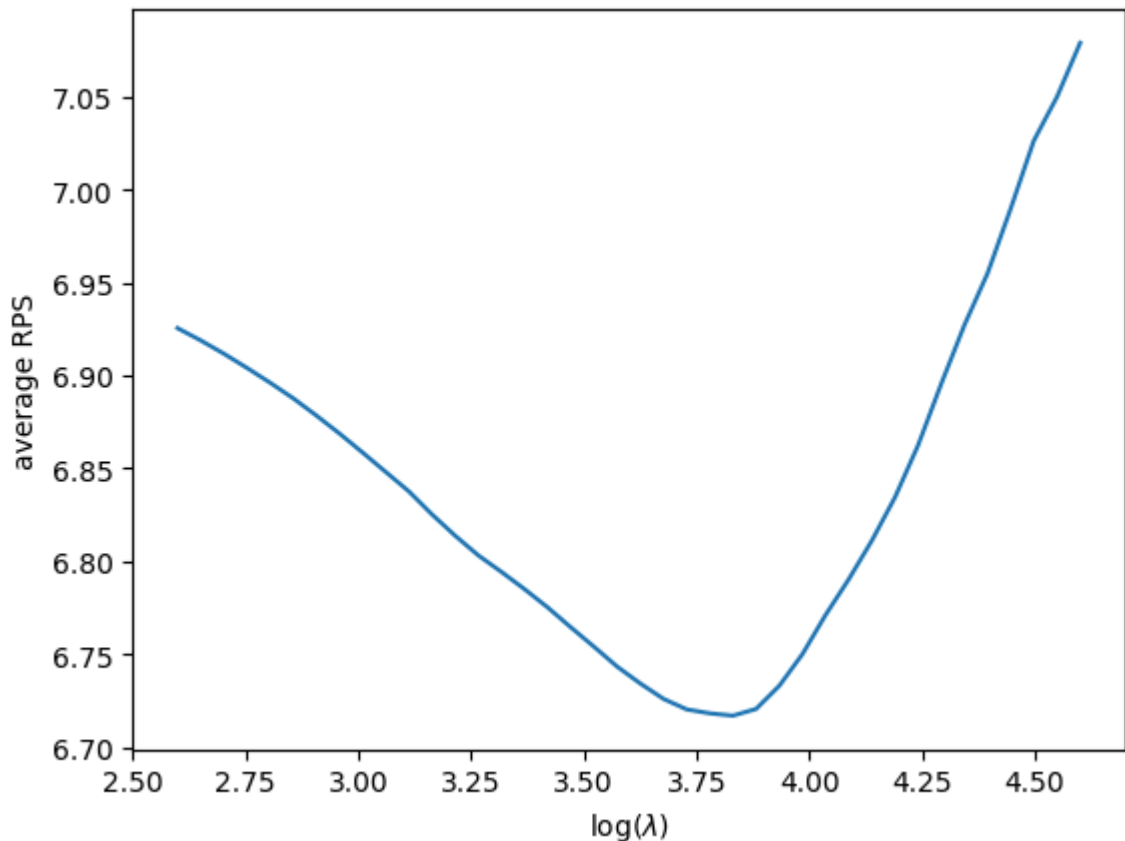
```

```

Test for $\lambda$ = 398.1072, average RPS = 6.9255 , iteration number: 1/40
Test for $\lambda$ = 448.0042, average RPS = 6.9189 , iteration number: 2/40
Test for $\lambda$ = 504.1550, average RPS = 6.9117 , iteration number: 3/40
Test for $\lambda$ = 567.3436, average RPS = 6.9041 , iteration number: 4/40
Test for $\lambda$ = 638.4519, average RPS = 6.8961 , iteration number: 5/40
Test for $\lambda$ = 718.4727, average RPS = 6.8877 , iteration number: 6/40
Test for $\lambda$ = 808.5229, average RPS = 6.8784 , iteration number: 7/40
Test for $\lambda$ = 909.8596, average RPS = 6.8687 , iteration number: 8/40
Test for $\lambda$ = 1023.8973, average RPS = 6.8585 , iteration number: 9/40
Test for $\lambda$ = 1152.2281, average RPS = 6.8480 , iteration number: 10/40
Test for $\lambda$ = 1296.6433, average RPS = 6.8376 , iteration number: 11/40
Test for $\lambda$ = 1459.1588, average RPS = 6.8252 , iteration number: 12/40
Test for $\lambda$ = 1642.0433, average RPS = 6.8137 , iteration number: 13/40
Test for $\lambda$ = 1847.8498, average RPS = 6.8032 , iteration number: 14/40
Test for $\lambda$ = 2079.4511, average RPS = 6.7943 , iteration number: 15/40
Test for $\lambda$ = 2340.0803, average RPS = 6.7850 , iteration number: 16/40
Test for $\lambda$ = 2633.3757, average RPS = 6.7751 , iteration number: 17/40
Test for $\lambda$ = 2963.4314, average RPS = 6.7644 , iteration number: 18/40
Test for $\lambda$ = 3334.8548, average RPS = 6.7540 , iteration number: 19/40
Test for $\lambda$ = 3752.8307, average RPS = 6.7433 , iteration number: 20/40
Test for $\lambda$ = 4223.1939, average RPS = 6.7342 , iteration number: 21/40
Test for $\lambda$ = 4752.5104, average RPS = 6.7260 , iteration number: 22/40
Test for $\lambda$ = 5348.1691, average RPS = 6.7203 , iteration number: 23/40
Test for $\lambda$ = 6018.4850, average RPS = 6.7182 , iteration number: 24/40
Test for $\lambda$ = 6772.8153, average RPS = 6.7170 , iteration number: 25/40
Test for $\lambda$ = 7621.6900, average RPS = 6.7206 , iteration number: 26/40
Test for $\lambda$ = 8576.9590, average RPS = 6.7330 , iteration number: 27/40
Test for $\lambda$ = 9651.9571, average RPS = 6.7500 , iteration number: 28/40
Test for $\lambda$ = 10861.6908, average RPS = 6.7711 , iteration number: 29/40
Test for $\lambda$ = 12223.0472, average RPS = 6.7903 , iteration number: 30/40
Test for $\lambda$ = 13755.0299, average RPS = 6.8111 , iteration number: 31/40
Test for $\lambda$ = 15479.0246, average RPS = 6.8345 , iteration number: 32/40
Test for $\lambda$ = 17419.0971, average RPS = 6.8625 , iteration number: 33/40
Test for $\lambda$ = 19602.3298, average RPS = 6.8956 , iteration number: 34/40
Test for $\lambda$ = 22059.1993, average RPS = 6.9271 , iteration number: 35/40
Test for $\lambda$ = 24824.0018, average RPS = 6.9550 , iteration number: 36/40
Test for $\lambda$ = 27935.3326, average RPS = 6.9898 , iteration number: 37/40
Test for $\lambda$ = 31436.6238, average RPS = 7.0264 , iteration number: 38/40
Test for $\lambda$ = 35376.7513, average RPS = 7.0499 , iteration number: 39/40
Test for $\lambda$ = 39810.7171, average RPS = 7.0789 , iteration number: 40/40

```

Best error : 6.7170, Best \$\lambda\$: 6772.815277



```
In [17]: betas=minimize_ls_lasso(x_train_augmente,y_train,best_lambda)
```

Now we can see which are the best features selected by LASSO algorithm and plot a feature importance histogram

```
In [16]: importance_df = pd.DataFrame({
    'Feature': statistics_names,
    'Beta_Value': betas[1:],
    'Absolute_Importance': np.abs(betas[1:]),
    'Relative_Importance' : np.abs(betas[1:])/np.max(abs(betas[1:]))
})

importance_df = importance_df.sort_values(by='Absolute_Importance', ascending=False)
print(f"Top 15 Features (lambda = {best_lambda:.0f})")
display(importance_df.head(15))

important_features = importance_df.head(12)['Feature']

threshold = 0.01
df_plot = importance_df[importance_df['Relative_Importance'] > threshold].copy()

df_plot = df_plot.sort_values(by='Relative_Importance', ascending=True)

plt.figure(figsize=(10, max(6, len(df_plot) * 0.4)))

bars = plt.barh(df_plot['Feature'], df_plot['Relative_Importance'], color='#3498db')

for bar in bars:
    largeur = bar.get_width()
```

```

plt.text(largeur + 0.01, bar.get_y() + bar.get_height()/2,
         f'{largeur:.2f}',
         va='center', ha='left', fontsize=10, fontweight='bold', color='#2c3e50')

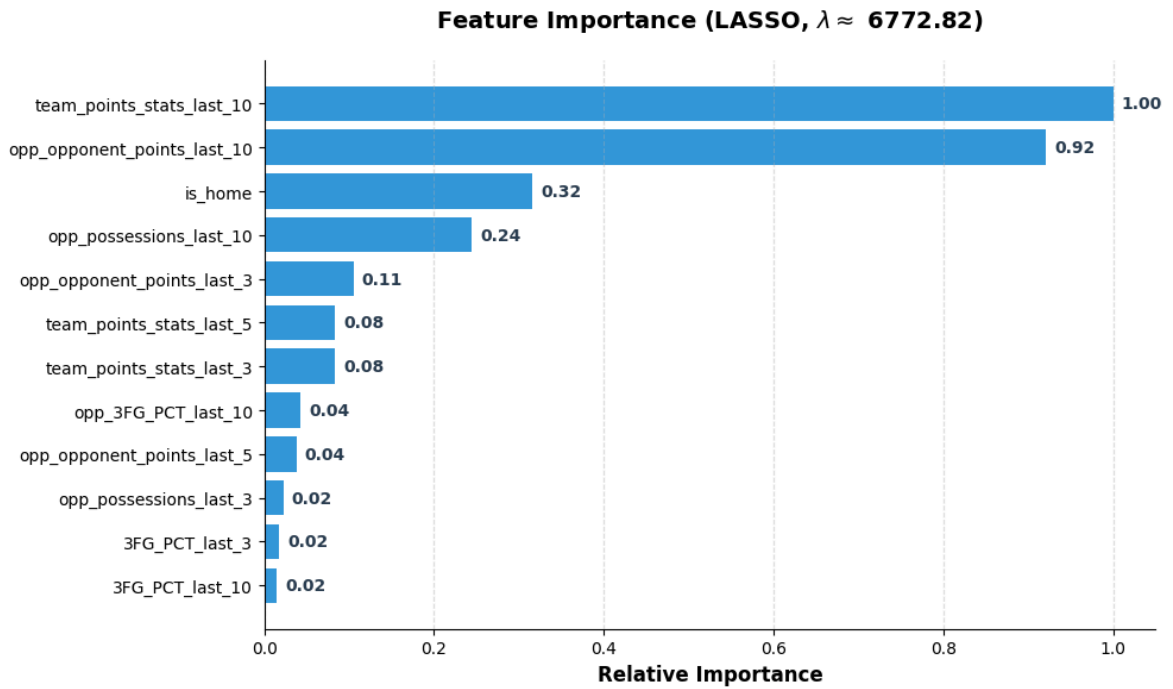
plt.xlabel('Relative Importance', fontsize=12, fontweight='bold')
plt.title(fr'Feature Importance (LASSO, $\lambda$ approx$ {best_lambda:.2f})', f
plt.grid(axis='x', linestyle='--', alpha=0.4)
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

```

Top 15 Features ($\lambda = 6773$)

	Feature	Beta_Value	Absolute_Importance	Relative_Importance
2	team_points_stats_last_10	2.371875	2.371875	1.000000
59	opp_opponent_points_last_10	2.184159	2.184159	0.920857
60	is_home	0.749287	0.749287	0.315905
41	opp_possessions_last_10	0.580261	0.580261	0.244642
57	opp_opponent_points_last_3	0.249093	0.249093	0.105019
1	team_points_stats_last_5	0.198215	0.198215	0.083569
0	team_points_stats_last_3	0.197515	0.197515	0.083274
47	opp_3FG_PCT_last_10	-0.101548	0.101548	0.042813
58	opp_opponent_points_last_5	0.090029	0.090029	0.037957
39	opp_possessions_last_3	0.053244	0.053244	0.022448
15	3FG_PCT_last_3	-0.042817	0.042817	0.018052
17	3FG_PCT_last_10	-0.036103	0.036103	0.015221
46	opp_3FG_PCT_last_5	-0.001069	0.001069	0.000451
44	opp_FG_PCT_last_10	-0.000961	0.000961	0.000405
36	opp_DRTG_last_3	0.000940	0.000940	0.000396



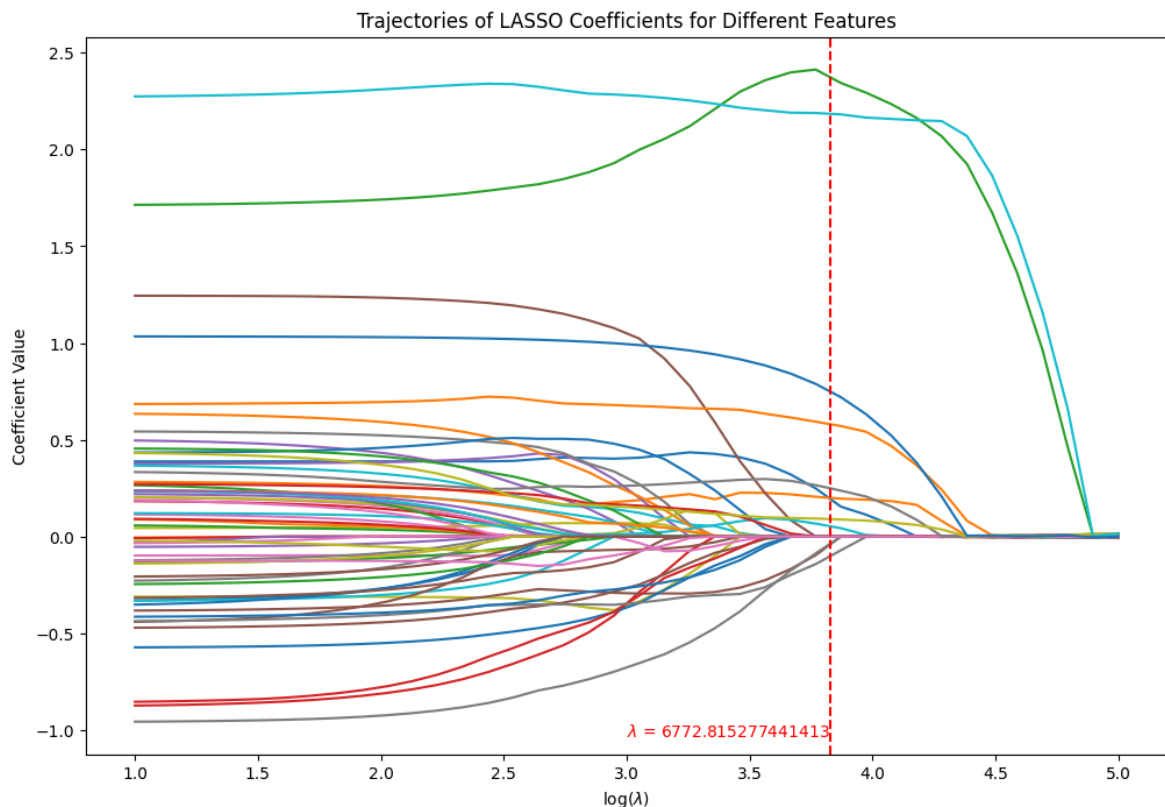
It is always interesting to plot the feature coefficients as a function of lambda, to see the shrinkage operated by LASSO.

```
In [18]: lambdas=np.logspace(1,5,40)
coeff_trajectories = []
for lam in lambdas:
    # Run LASSO regression
    beta_lasso = minimize_ls_lasso(x_train_augmente, y_train, lamdb=lam, n_iters
                                step_size=1e-7,
                                c_huber=1e-5)
    coeff_trajectories.append(beta_lasso[1:])

coeffs=np.vstack(coeff_trajectories)

# plot the trajectores of LASSO coefficients using log-scale
import matplotlib.pyplot as plt
plt.figure(figsize=(12, 8))
for i in range(coeffs.shape[1]):
    plt.plot(np.log10(lambdas), coeffs[:, i], label=f'Feature {i + 1}')

plt.xlabel('log( $\lambda$ )')
plt.ylabel('Coefficient Value')
plt.title('Trajectories of LASSO Coefficients for Different Features')
plt.axvline(x=np.log10(best_lambda), color='red', linestyle='--')
plt.text(np.log10(best_lambda), 0.02, f'  $\lambda = \{best\_lambda\}$  ',
        color='red',
        transform=plt.gca().get_xaxis_transform(),
        ha='right',
        va='bottom',
        fontsize=10)
plt.show()
```



To determine whether the model is efficient, we compare its RPS with naive approach considering team point distribution as normal with mean the mean of the 10 previous team points and standard deviation the MSE of residuals.

We compare the score on the test set.

```
In [18]: naive_errors_train = df_final_without_na['team_points'].values[:index_max] - df_
naive_sigma = np.std(naive_errors_train)
naive_means = df_final_without_na['team_points_stats_last_10'].values[index_max:
naive_cdf = discretize_normal_cdf(naive_means, naive_sigma, support_max=200)
naive_rps = ranked_probability_score(naive_cdf, y_test, support=np.arange(201))

model_means = x_test_augmente@betas
past_model = x_train_augmente@betas
residuals = past_model-y_train
model_sigma = np.std(residuals)
model_cdf = discretize_normal_cdf(model_means,model_sigma, support_max=200)
model_rps = ranked_probability_score(model_cdf, y_test, support=np.arange(201))

print(f"RPS of naive model: {naive_rps:.4f}")
print(f"RPS of LASSO model: {model_rps:.4f}")
print(f"Skill Score : {(1 -model_rps/naive_rps) *100:.4f}%")
```

RPS of naive model: 7.3309

RPS of LASSO model: 6.8463

Skill Score : 6.6099%

```
In [ ]: important_index = [statistics_names.index(feature) for feature in important_feat

# x_train_Tree = x_train_standardize
```

```
x_train_Tree = x_train_standardize[:,important_index]
x_test_Tree = x_test_standardize[:,important_index]
```

Here we build random forests. To do so, for more clarity, we use object-oriented programming aspect of python. We define the following classes :

- Class Node, which are the structures of trees in forests. They are defined by threshold and dimension to decide how to split data.
- Class Tree, which implement the methods to train decision trees. A tree is only a linked list of nodes, hence we can define them by the head of the list
- Class RandomForest, which implement the methods to train random forest i.e with both bootstrap aggregating and feature bagging. Random Forests are just list of such trained trees trained on random samples and selected features (at each split)

```
In [20]: class Node:
def __init__(self, dimension_number=None, treshhold=None):
    self.dimension_number = dimension_number
    self.treshhold = treshhold
    self.left = None
    self.right = None
    self.probability_distribution = None

def add_left(self, child_node):
    self.left = child_node

def add_right(self, child_node):
    self.right = child_node

def get_left_son(self):
    return self.left

def get_right_son(self):
    return self.right
```

```
In [21]: class Tree:

def __init__(self,max_depth,min_std,max_features):
    self.head = None
    self.max_depth=max_depth
    self.min_std = min_std
    self.max_features=max_features #for feature bagging

def training(self,x_train,y_train):
    features = np.arange(x_train.shape[1])
    self.head= self.build_Nodes(x_train,y_train,features)

def build_Nodes(self,x_train,y_train,features,depth=0) :

    if np.std(y_train) < self.min_std or y_train.shape[0] < 2 or depth > self
        leaf = Node()
        leaf.probability_distribution = self.point_distribution(y_train)
        return leaf

    random_features=np.random.choice(features,self.max_features,replace=False,
column, treshhold = self.best_split(x_train,y_train,random_features)
```



```

        if column is None:
            leaf = Node()
            leaf.probability_distribution = self.point_distribution(y_train)
            return leaf

    x_left, x_right, y_left, y_right = self.split(x_train, y_train, column,

    left_child = self.build_Nodes(x_left,y_left,features,depth+1)
    right_child = self.build_Nodes(x_right,y_right,features,depth+1)

    parent = Node(column,treshold)
    parent.add_left(left_child)
    parent.add_right(right_child)

    return parent

def get_distributions(self,x,support):
    sample_size= x.shape[0]
    distributions = np.zeros((sample_size, len(support))) #initializing dist
    for k in range(sample_size) :
        data_point = x[k]
        current_Node = self.head

        while current_Node.probability_distribution is None:
            column = current_Node.dimension_number
            treshold = current_Node.treshold
            if data_point[column] < treshold:
                current_Node = current_Node.get_left_son()
            else:
                current_Node = current_Node.get_right_son()

        probability_distribution = current_Node.probability_distribution
        for point,proba in probability_distribution.items():
            index = np.where(point == support) #for general supports.
            if len(index) > 0:
                distributions[k, index] = proba
    return distributions

def split(self,x, y, column, treshold):
    mask_index = x[:, column] < treshold
    x_left = x[mask_index]
    x_right = x[~mask_index]
    y_left = y[mask_index]
    y_right = y[~mask_index]
    return x_left, x_right, y_left, y_right

def best_split(self,x_train, y_train,random_features):
    n = y_train.shape[0]
    mini_cost = np.inf
    current_best_split = (None, None)

    for column in random_features:
        for treshold in np.unique(x_train[:, column]):

```

```

        x_left, x_right, y_left, y_right = self.split(x_train, y_train,

        if len(y_left) == 0 or len(y_right) == 0:
            continue
        r1 = x_left.shape[0] / n
        r2 = x_right.shape[0] / n
        cost = r1 * np.std(y_left) + r2 * np.std(y_right)

        if cost < mini_cost:
            mini_cost = cost
            current_best_split = (column, threshold)

    return current_best_split

def point_distribution(self,y):
    n = y.shape[0]
    probability_distribution = {}
    for point in np.unique(y):
        count = np.sum(y == point)
        probability_distribution[point] = count / n

    return probability_distribution

```

```

In [ ]: from joblib import Parallel, delayed

def train_single_tree(max_depth, min_std, m, x_train, y_train):
    n_samples = y_train.shape[0]
    model = Tree(max_depth, min_std, m)
    random_index = np.random.choice(n_samples, size=n_samples, replace=True) #1/
    model.training(x_train[random_index, :], y_train[random_index])

    return model

class RandomForest :
    def __init__(self,B,m,max_depth,min_std,n_jobs=-1):
        self.trees = []
        self.B=B
        self.m=m
        self.max_depth=max_depth
        self.min_std=min_std
        self.n_jobs=n_jobs

    def train_forest(self,x_train,y_train):
        self.trees = Parallel(n_jobs=self.n_jobs)(
            delayed(train_single_tree)(
                self.max_depth, self.min_std, self.m, x_train, y_train
            )
            for _ in range(self.B)
        )

    def get_distribution(self,x,support):
        trees_predictions= []
        for model in self.trees:
            distributions = model.get_distributions(x,support)
            trees_predictions.append(distributions)

        predictions = np.mean(trees_predictions,axis=0)

```

```
return predictions
```

```
In [23]: def T_fold_cross_validation_forest(x_train,y_train,subsets_indexes,min_std) :
errors=[]
for k in range(1,len(subsets_indexes)-1):
    subset_index=subsets_indexes[k]
    next_subset_index=subsets_indexes[k+1]
    x_train_t=x_train[:subset_index,:]
    y_train_t=y_train[:subset_index]
    x_val=x_train_Tree[subset_index:next_subset_index,:]
    y_val=y_train[subset_index:next_subset_index]
    B=15
    depth = 8
    forest=RandomForest(B,x_train.shape[1]-1,depth,min_std)
    forest.train_forest(x_train_t,y_train_t)
    support=np.arange(200)
    predicted_distribution = forest.get_distribution(x_val,support)
    error= ranked_probability_score(predicted_distribution, y_val, support)
    errors.append(error)

return np.mean(errors)
```

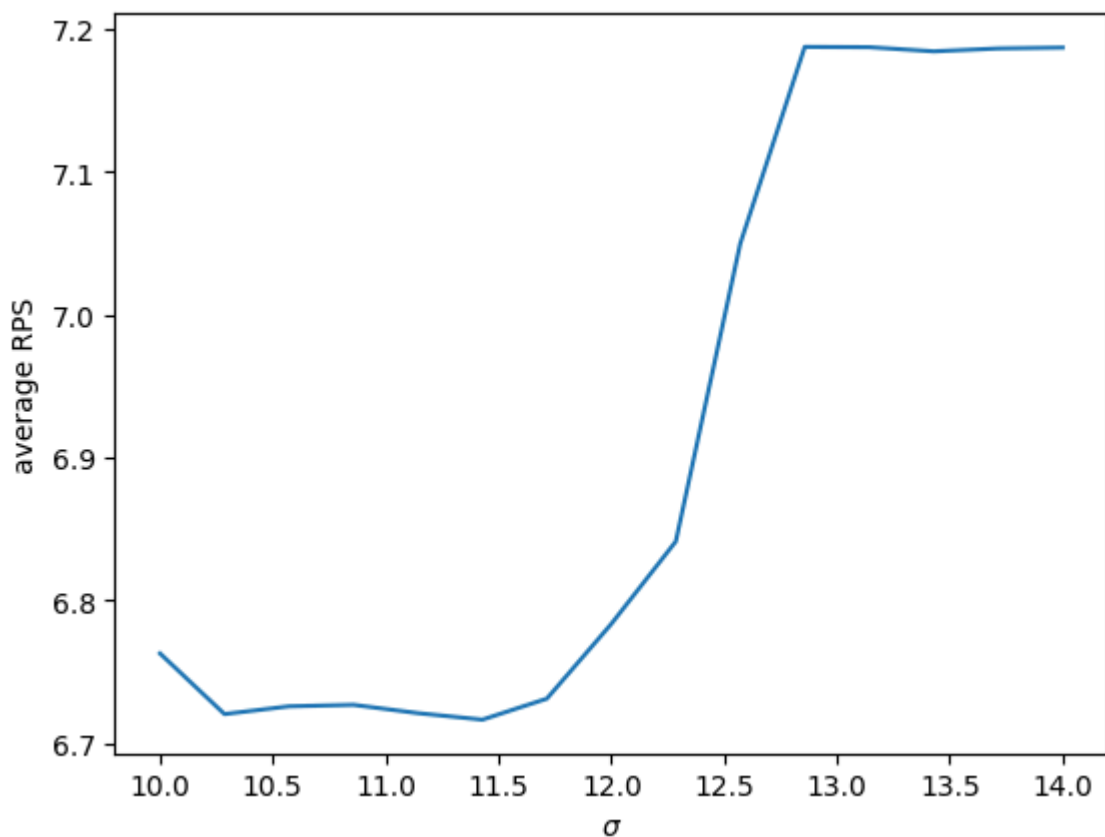
```
In [24]: folds_number=5
subsets_indexes=np.linspace(0,x_train_Tree.shape[0],folds_number,dtype=int)
sigmas = np.linspace(10, 14.0,15)

best_lambda,losses = record_loss_curve_cross_validation(x_train_Tree,y_train,sig
plt.plot(sigmas,losses)
plt.xlabel(f'$\sigma$')
plt.ylabel("average RPS")
plt.show()
```

```
<>:5: SyntaxWarning: invalid escape sequence '\l'
<>:7: SyntaxWarning: invalid escape sequence '\s'
<>:5: SyntaxWarning: invalid escape sequence '\l'
<>:7: SyntaxWarning: invalid escape sequence '\s'
C:\Users\leand\AppData\Local\Temp\ipykernel_25420\640774631.py:5: SyntaxWarning:
invalid escape sequence '\l'
    best_lambda,losses = record_loss_curve_cross_validation(x_train_Tree,y_train,si
gmas,T_fold_cross_validation_forest,subsets_indexes,'$\lambda$')
C:\Users\leand\AppData\Local\Temp\ipykernel_25420\640774631.py:7: SyntaxWarning:
invalid escape sequence '\s'
    plt.xlabel(f'$\sigma$')
```

Test for $\lambda = 10.0000$, average RPS = 6.7629 , iteration number: 1/15
 Test for $\lambda = 10.2857$, average RPS = 6.7205 , iteration number: 2/15
 Test for $\lambda = 10.5714$, average RPS = 6.7259 , iteration number: 3/15
 Test for $\lambda = 10.8571$, average RPS = 6.7269 , iteration number: 4/15
 Test for $\lambda = 11.1429$, average RPS = 6.7211 , iteration number: 5/15
 Test for $\lambda = 11.4286$, average RPS = 6.7165 , iteration number: 6/15
 Test for $\lambda = 11.7143$, average RPS = 6.7312 , iteration number: 7/15
 Test for $\lambda = 12.0000$, average RPS = 6.7836 , iteration number: 8/15
 Test for $\lambda = 12.2857$, average RPS = 6.8413 , iteration number: 9/15
 Test for $\lambda = 12.5714$, average RPS = 7.0497 , iteration number: 10/15
 Test for $\lambda = 12.8571$, average RPS = 7.1872 , iteration number: 11/15
 Test for $\lambda = 13.1429$, average RPS = 7.1870 , iteration number: 12/15
 Test for $\lambda = 13.4286$, average RPS = 7.1841 , iteration number: 13/15
 Test for $\lambda = 13.7143$, average RPS = 7.1861 , iteration number: 14/15
 Test for $\lambda = 14.0000$, average RPS = 7.1867 , iteration number: 15/15

Best error : 6.7165, Best λ : 11.428571



```

In [25]: forest=RandomForest(15,x_train_Tree.shape[1]-1,8,11.428571)
forest.train_forest(x_train_Tree,y_train)
support=np.arange(200)
predicted_distribution = forest.get_distribution(x_test_Tree,support)
error= ranked_probability_score(predicted_distribution,y_test, support)

print(f"RPS of naive model: {naive_rps:.4f}")
print(f"RPS of LASSO model: {model_rps:.4f}")
print(f"RPS of forest model: {error:.4f}")
print(f"Skill Score : {(1 -error/naive_rps) *100:.4f}%")
  
```

```
RPS of naive model: 7.3309  
RPS of LASSO model: 6.8463  
RPS of forest model: 6.8971  
Skill Score : 5.9170%
```

In []: